Explainable Exoplanet Classification and Ranking System with Database-Backed Interactive Dashboard

IT Workshop III - Course Project Report

Name: Shaeel Santosh Hasepatil | Roll No.: 2025BCS0133

Abstract

The Kepler telescope found thousands of dips in starlight that could be planets passing in front of a star. Many of these are not planets but false positives, for example two stars eclipsing each other. In this project I use the NASA Kepler Objects of Interest (KOI) table (9,564 signals) to build a model that separates real planets from false positives, ranks the undecided CANDIDATE signals, explains its decisions, saves predictions in a SQLite database and has a small Gradio app.

This project extends my earlier capstone classifier. After fixing the cross-validation, the honest baseline was a macro F1 of 0.749. Adding back diagnostic features raised it to 0.805. A separate CONFIRMED vs FALSE POSITIVE model (XGBoost) then reached a test macro F1 of 0.973 on stars it had never seen, and I used it to rank 1,977 unresolved candidates.

1. Introduction

When a planet passes in front of its star, the star gets a little dimmer. Kepler recorded such dips and NASA labelled each signal as **CONFIRMED** (a planet), **FALSE POSITIVE** (not a planet) or **CANDIDATE** (not decided yet). Many dips are caused by eclipsing binary stars or by a nearby star whose light mixes in. Checking each signal by hand is slow, so a model that can vet signals automatically is useful.

Problem statement: given the measurements of a Kepler signal, predict whether it is a planet or a false positive, rank the undecided candidates, explain the prediction, and make the system easy to use.

Objectives:

- Evaluate models properly, without data leakage.
- Check if adding back diagnostic features (centroid offset, odd-even depth, event statistics) helps.
- Build a CONFIRMED vs FALSE POSITIVE model and compare Random Forest, Gradient Boosting and XGBoost.
- Rank the unresolved candidates by planet probability.
- Explain the model with permutation importance and SHAP.
- Store predictions in SQLite and build a Gradio app.

This work builds on my earlier capstone for the BRICS Astronomy / IDIA course (github.com/Shaeel-Hasepatil/exoplanet-disposition-classification), which trained Logistic Regression, Random Forest and Gradient Boosting on the three classes.

2. Dataset

The data is the KOI cumulative table from the NASA Exoplanet Archive, loaded with the *astroquery* library. It has 9,564 rows and 155 columns. The target is *koi_disposition*.

Class	Count	Share
FALSE POSITIVE	4,839	50.6%
CONFIRMED	2,748	28.7%
CANDIDATE	1,977	20.7%

Table 1: Class counts.

Cleaning: I removed empty columns, identifiers and columns that give away the answer (such as NASA's own score and the false-positive flags). Missing values are filled with the column median. The first feature set had 51 columns.

Important features: transit depth, duration and impact parameter describe the dip. Planet radius and star properties come from those. Signal-to-noise ratio tells how strong the dip is. The *_err* columns are measurement uncertainties. The diagnostic features are the centroid offsets (*koi_dicco_msky*, *koi_dikco_msky*, *koi_fwm_stat_sig*), the odd-even depth test (*koi_bin_oedp_sig*) and the event statistics (*koi_max_mult_ev*, *koi_max_sngle_ev*). They help detect eclipsing binaries and contamination.

3. Methodology

3.1 First models

I first trained Logistic Regression, Random Forest and Gradient Boosting on the 51 features. On one split the macro F1 was about 0.76 and the CANDIDATE class was weak (F1 about 0.56). I use macro F1 because it gives every class equal weight, so a model cannot look good by favouring the biggest class.

3.2 Fixing the evaluation

The first cross-validation had three problems:

- **Folds were not shuffled.** The table is sorted by star ID, so each fold covered a different part of the data and scores jumped around.
- **Same star in train and test.** One star can have several signals. I now use StratifiedGroupKFold with the star ID (*kepid*) as the group, so a star is never split. The ID is only used for grouping, not as a feature.
- **Imputation before splitting.** I put the imputer and model inside a scikit-learn Pipeline so the medians are learned only from training data in each fold.

3.3 Restoring the diagnostic features

Eight diagnostic columns had been left out in the first version. I added them back, which gives 59 features, and compared 51 vs 59 features with the same Random Forest and the same grouped splits (an ablation study).

3.4 Vetting model and model comparison

CANDIDATE signals have no final answer, so I trained the vetting model only on resolved signals: CONFIRMED (planet) vs FALSE POSITIVE (7,587 rows). About 20% of the stars were kept as a test set (6,082 training rows and 1,505 test rows, no star in both). I compared Random Forest, Gradient Boosting and XGBoost, each tuned with RandomizedSearchCV (8 random settings, 3-fold grouped CV on the training stars). Class imbalance was handled with balanced sample weights. The best model was chosen by cross-validation score, and the test set was used only once at the end.

3.5 Ranking, explainability, database and app

- **Ranking:** the chosen model was refitted on all resolved rows and used to give each CANDIDATE a planet probability.
- **Explainability:** permutation importance (shuffle a feature and see how much the score drops) and SHAP values (each feature's contribution to a prediction). I also retrained without the *_err* columns to see if they act as a shortcut.
- **Database:** SQLite with two tables, *model_info* (model name, scores, date) and *predictions* (signal name, star ID, planet probability, label, source, time).
- **App:** a Gradio form with 10 inputs: the 10 most important features from the permutation importance (centroid offsets, planet radius, event statistics, planet-star distance, duration, star metallicity and star density). It has three example rows to click. The app is also saved as a standalone file (app_files/app.py) with the trained model, so it can be run locally or hosted. It shows the planet probability and the top 5 reasons, and saves each prediction to SQLite. Empty or missing features are filled with the training median.

4. Implementation

Everything is in one Jupyter Notebook run on Google Colab, using Python with pandas, NumPy, Matplotlib, Seaborn, scikit-learn, XGBoost, SHAP, sqlite3 and Gradio. The main parts are the Pipeline with grouped cross-validation (Sections 14-15), the tuned model search (Section 16), the ranking (Section 17), SHAP and permutation importance (Section 18), the SQLite code (Section 19) and the Gradio function (Section 20).

5. Results

5.1 Effect of the evaluation fix

Split type	Mean macro F1	Std	Min fold	Max fold
No shuffle	0.7072	0.0585	0.6177	0.7739
Shuffled	0.7550	0.0080	0.7440	0.7663
Shuffled, grouped by star	0.7492	0.0104	0.7375	0.7648

Table 2: Random Forest, base features, 5-fold CV.

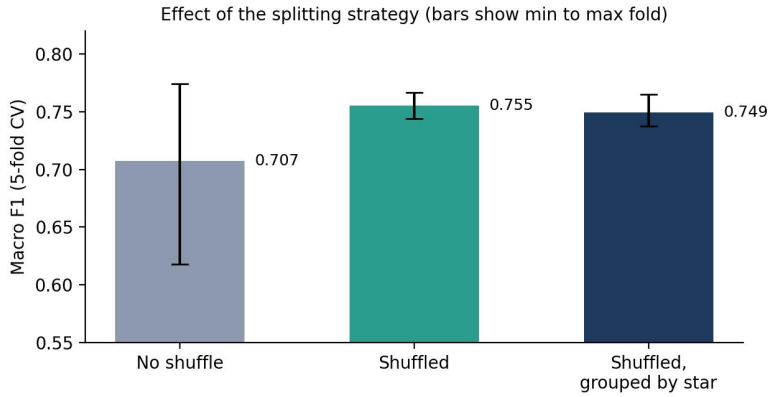


Figure 1: Macro F1 for the three split types.

Shuffling reduced the standard deviation from 0.0585 to 0.0080, so the big swings came from the ordered folds and not from the model. Grouping by star lowered the score a little (0.755 to 0.749), which means a small part of the earlier score came from the same star being in both train and test. So 0.749 is my honest baseline.

5.2 Effect of the restored features

Features	n	Macro F1	F1 CANDIDATE	F1 CONFIRMED	F1 FALSE POS.
Base	51	0.7492	0.5445	0.8548	0.8490
Base + restored	59	0.8054	0.6282	0.8924	0.8964

Table 3: Ablation result (3-class Random Forest, grouped 5-fold CV).

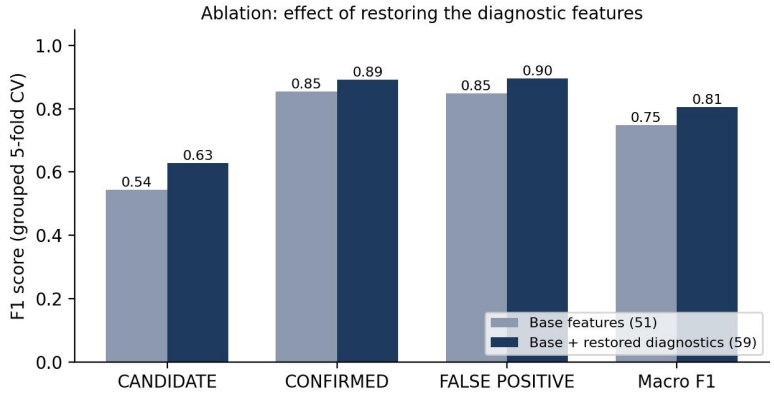


Figure 2: Per-class F1 with and without the restored features.

Macro F1 went from 0.749 to 0.805. All three classes improved and CANDIDATE improved the most (+0.084). This makes sense because these features directly detect eclipsing binaries and contamination. One caution: Kepler's own labels were partly made using these same diagnostics, so the model is partly learning the vetting rules.

5.3 Vetting model (CONFIRMED vs FALSE POSITIVE)

Model	CV macro F1	Test accuracy	Test macro F1	Test ROC-AUC
Random Forest	0.9670	0.9688	0.9656	0.9944
Gradient Boosting	0.9729	0.9714	0.9688	0.9952
XGBoost (selected)	0.9745	0.9754	0.9731	0.9950

Table 4: Tuned models on the held-out test stars.

XGBoost had the best cross-validation score, so it was selected. All three models are very close (test macro F1 between 0.966 and 0.973), so the features matter more than the algorithm. On the 1,505 test signals, XGBoost made 37 mistakes: 18 false positives accepted as planets and 19 planets rejected (956 and 512 were correct). The score is much higher than the 3-class result of 0.805 because the hard CANDIDATE class is not included, so the two numbers should not be compared directly.

5.4 Candidate ranking

The model scored 1,977 candidates. 487 have a planet probability of at least 0.9 and 677 have at least 0.5. The histogram below shows most candidates are at the two ends: about 950 are below 0.05 and about 420 are above 0.95. This ranking cannot be checked directly because these candidates have no true labels. It is only supported by the test result in 5.3.

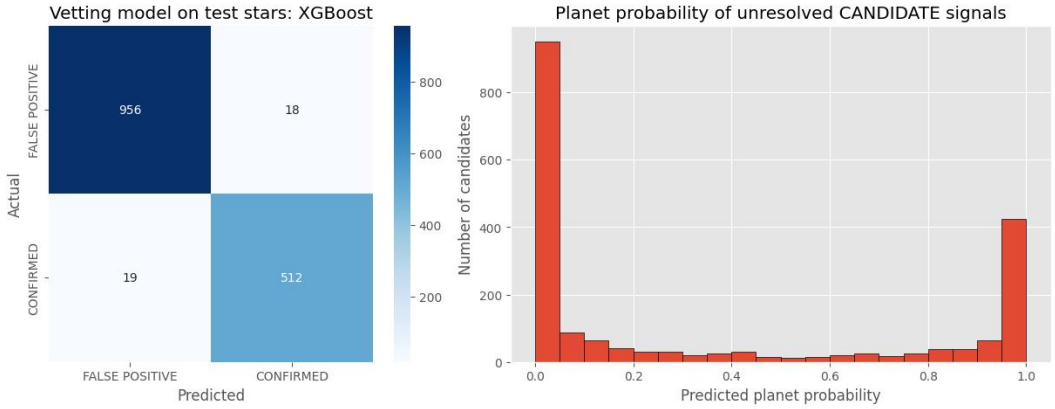


Figure 3: Confusion matrix on test stars (left) and planet probability of the candidates (right).

5.5 Explainability

Permutation importance and SHAP agree on the top features: *koi_dikco_msky* and *koi_dicco_msky* (centroid offsets), *koi_prad* (planet radius), *koi_fwm_stat_sig* and *koi_max_mult_ev*. Most of the top features are the restored diagnostics, which supports the ablation result.

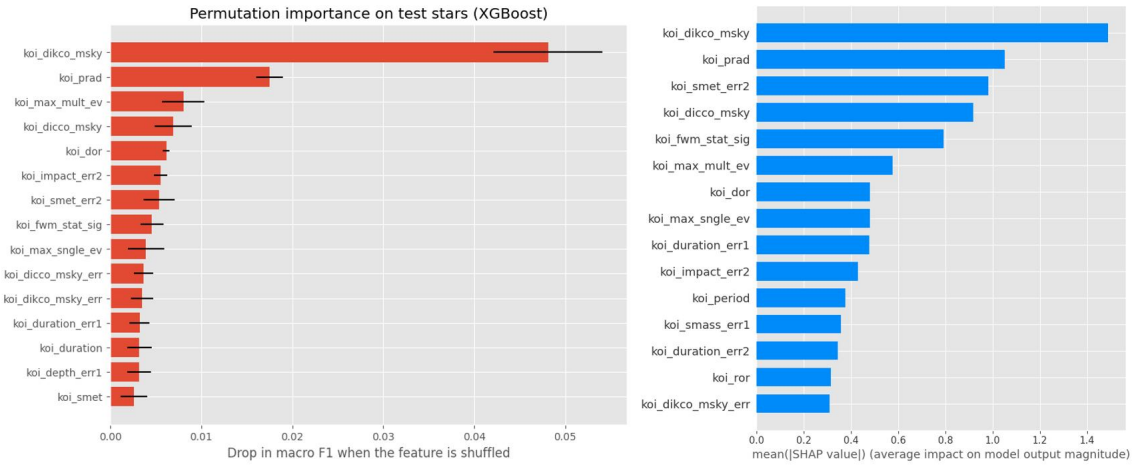


Figure 4: Permutation importance (left) and SHAP importance (right).

In the error-bar test, the test macro F1 was 0.9731 with all 59 features and 0.9680 without the 30 *_err* columns (29 features left). The drop is only 0.005, so the model does not depend on them.

But 6 of the top 15 permutation features are *_err* columns (for example *koi_smet_err2* is third in SHAP), so there is a small effect.

5.6 Database and app

The ranking is stored in SQLite (1,977 rows). SQL queries give, for example, the top candidates and the count per probability band:

Probability band	**0.9-1.0**	**0.7-0.9**	**0.5-0.7**	**0.3-0.5**	**0.0-0.3**
Candidates	487	118	72	94	1,206

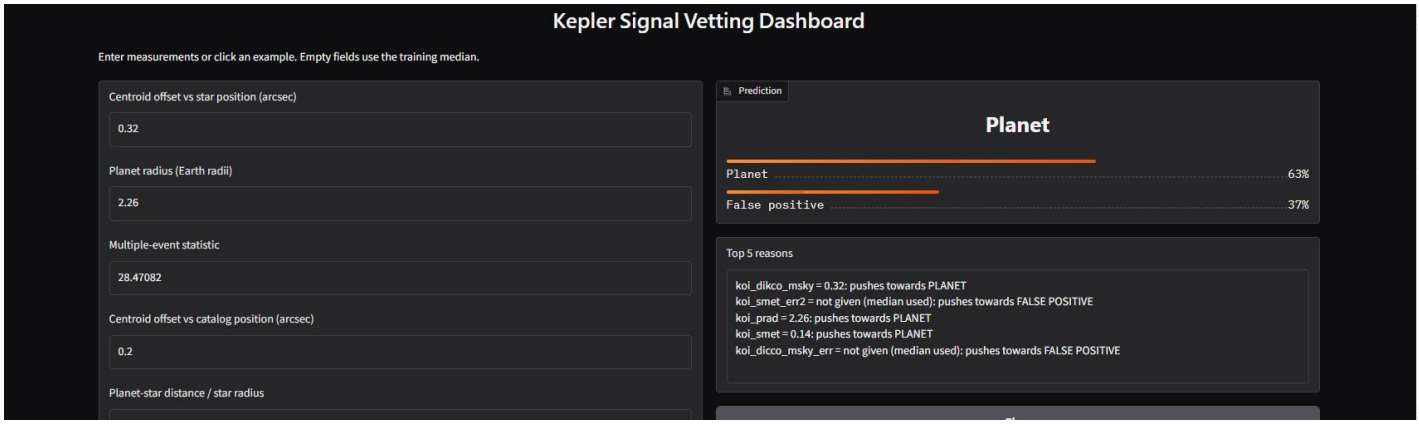
Table 5: Candidates per probability band (SQL query). 677 are labelled LIKELY PLANET and 1,300 LIKELY FALSE POSITIVE.

The app asks for only 10 fields, so I measured how much accuracy is lost. On the test stars I gave the model only the top-N features and filled the rest with medians:

Fields given	**5**	**10**	**15**	**20**	**29**	**All 59**
Test macro F1	0.3929	0.5273	0.6836	0.7246	0.8236	0.9731

Table 6: Test macro F1 when only the top-N features are given (29 = all features without the _err columns).

The score drops a lot with few fields, so the app is only an approximate demo of the model.



6. Limitations

- The labels come from Kepler's own vetting, which used some of these diagnostics. So the model is an automatic vetting aid, not an independent planet finder.
- The candidate ranking has no true labels to check against.
- I used one hold-out split and a small search (8 settings per model).
- The app only asks for 10 of the 59 features and fills the rest with medians. I tested this on the test stars: giving only the top 10 features gives a macro F1 of 0.527, compared with 0.973 with all 59 (Table 6). For example, the top-ranked candidate gets a planet probability of about 0.55 in the app but 1.0 from the full model. So the app is only an approximate demo.
- The results are only for Kepler data.

7. Conclusion

I turned my earlier three-class classifier into a vetting and ranking system. Fixing the evaluation gave an honest baseline of 0.749, and adding back the diagnostic features raised it to 0.805. The CONFIRMED vs FALSE POSITIVE model (XGBoost) reached a test macro F1 of 0.973 and ranked 1,977 candidates. SHAP and permutation importance show the model mainly uses the centroid and event diagnostics. Predictions are stored in SQLite and can be tried in the Gradio app.

Future work: let the app look up a KOI by name so it uses all 59 features, check the ranking against candidates that are later confirmed or rejected, and try the same approach on TESS data.

References

- NASA Exoplanet Archive, Kepler Objects of Interest Cumulative Table. https://exoplanetarchive.ipac.caltech.edu/
- McCauliff, S. D. et al. (2015). Automatic Classification of Kepler Planetary Transit Candidates. The Astrophysical Journal. https://arxiv.org/abs/1408.1496
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12.
- Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD.
- Lundberg, S. M. and Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. NeurIPS 30.
- Abid, A. et al. (2019). Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild. arXiv:1906.02569.